

Gridfall

Design Document

Group: Lag Wizards

Members: Ahmad Idris, Fong Vang, Jeffrey Cohn, Manas Patel, Tanner Gleason

1. Project Overview

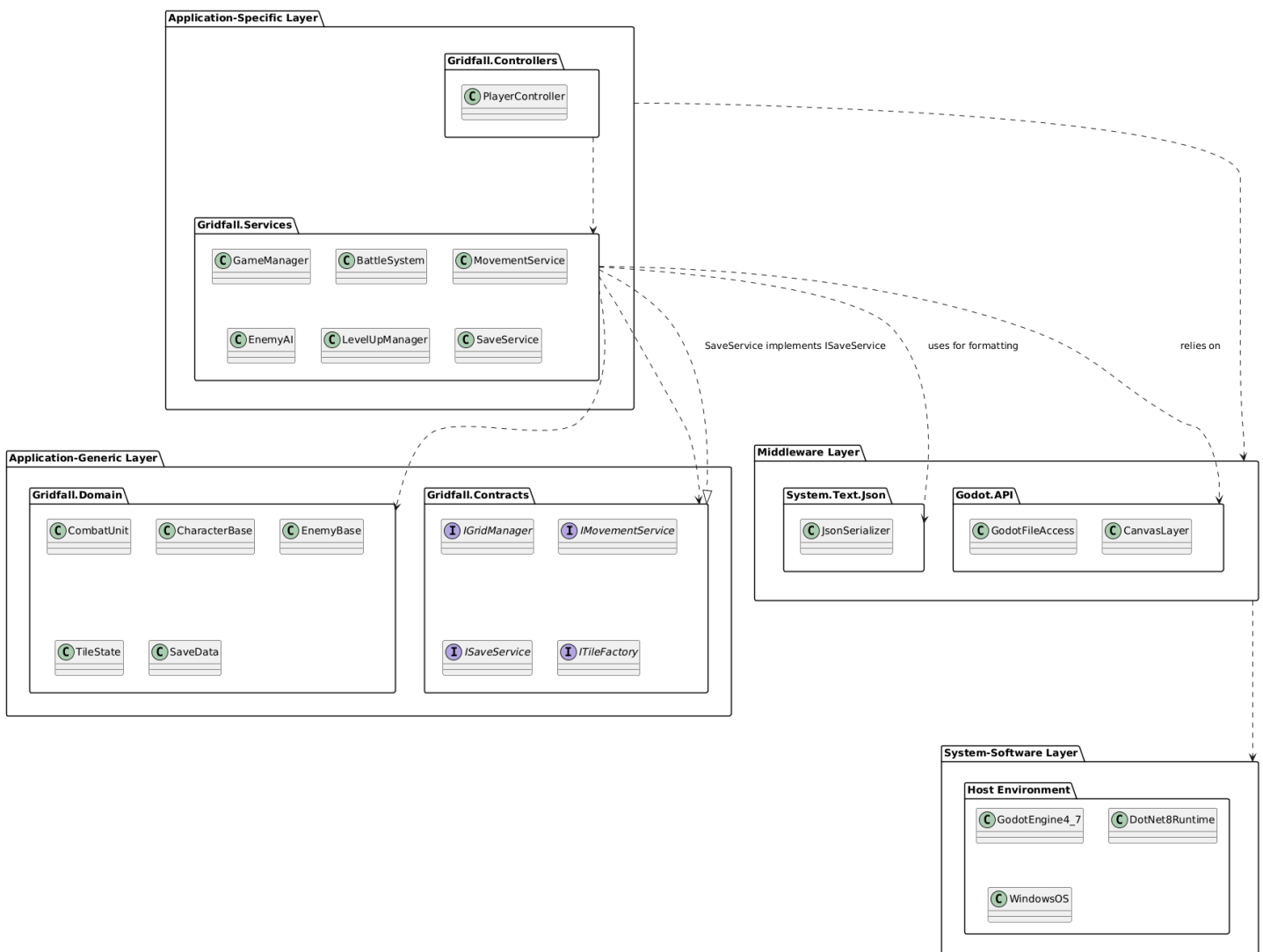
The project, Gridfall, aims to provide a video game gameplay experience similar to the video game franchise Fire Emblem. Though this would be available natively on Windows Computers, unlike Fire Emblem which is only natively available on Nintendo consoles. Stakeholders for such a project would be fans of the Fire Emblem franchise who own Windows computers but not Nintendo consoles. They would want such a game to be close to the gameplay of Fire Emblem and fun so they would have a viable game to play instead. By developing a game inspired by Fire Emblem for Windows computers, our project can fulfill this need for these stakeholders.

2. Architectural Overview

Gridfall is a game intended to be run by users locally on their Windows Computer. Given this, the most logical architectural style was a monolithic style as the game needs to run as a single easy to run program on the user's computer. We had considered using a layered architecture but this requires each layer to strictly speak to the layer directly beneath it. The logic and interaction between systems in a video game may require additional flexibility. The Model View Controller architecture then became the most viable option. The MVC architecture split of model, view, and controller, was intuitive for the game, with the model being domain objects like data and the rules of the game, view being the game's display including rendering and UI, and controller handling user input. MVC allowed for a logical separation of code roles and allowed for more flexibility in communication between pieces, since the view can be updated by both the model and controller, as opposed to the strictness of layered architecture.

2.1 Subsystem Architecture

The subsystem architecture for Gridfall is organized into four package layers to manage system dependencies and maintain low coupling. The Application-Specific Layer contains user controllers and game services. Below it, the Application-Generic Layer holds domain data models and system interfaces. Supporting these higher tiers is the Middleware Layer, which handles data formatting and classes to work with the engine. At the bottom, the System-Software Layer has the engine runtime, .NET environment, and operating system.



Each package in the architecture fulfills a specific responsibility in the application. The Gridfall.Controllers package handles user input, taking user actions to trigger character movement and menu interactions. The Gridfall.Services package manages core gameplay rules and logic, including combat, grid pathfinding, enemy decision making, character leveling, and save management. The Gridfall.Domain package defines the data structures for unit attributes, map tile data, and the save data structures. The Gridfall.Contracts package defines contracts through interfaces for classes, allowing services to interact without depending directly on the concrete class implementations. In the supporting layers, System.Text.Json manages data formatting by preparing the data for file storage, Godot.API provides visual user interface rendering containers and file handling tools, and Host Environment provides the Godot Engine, .NET environment, and Windows operating system required to run the compiled application locally.

Two primary architectural styles are applied within this subsystem layout. First, the Model-View-Controller (MVC) style separates user input handling, domain data and game rules, and visual interface rendering. As established in Section 2, MVC provides the flexibility needed for video games by allowing both input controllers and data models to update views. Also, the Observer pattern is implemented through the central static Events class, where it will automatically notify observers across the system whenever state changes occur. For example, when a unit dies, calling Events.EmitEnemyDied() automatically notifies all subscribed observers, such as GameManager and UI display controllers. This pattern was chosen to promote loose coupling, and prevent code dependencies by allowing game entities to broadcast state changes without direct references to UI elements or managers.

2.2 Deployment Architecture

This software will run on a single processor.

2.3 Persistent Data Storage

Gridfall requires persistent data storage to retain player character progress, accumulated currency, and inventory items across different play sessions. The information that must be stored includes the player's coin balance, as well as detailed statistics for each character including their: name, level, current health, combat statistics, accumulated experience points, equipped armor information, equipped weapon information, and a list of inventory items. Storing this information ensures that when the user exits the game, their progress is saved to disk and can be restored when loading a save slot later.

The approach chosen for storing persistent data is a flat file format using JSON. The save files are stored locally on the user's computer using Godot's user file system with the naming format `user://savegame_{slot}.json`, which maps on Windows to

`%APPDATA%\Godot\app_userdata\Gridfall\savegame_{slot}.json`. The file format represents a single root JSON object containing a `Coins` integer property and a `Characters` array property.

Inside the array, each element is a JSON object representing a character's data fields formatted as standard key-value pairs. Data handling is managed by `SaveService.cs`, which uses

`System.Text.Json` to format in-memory C# data objects into JSON text when saving, and read the JSON text back into memory objects when loading a save slot.

2.4 Global Control Flow

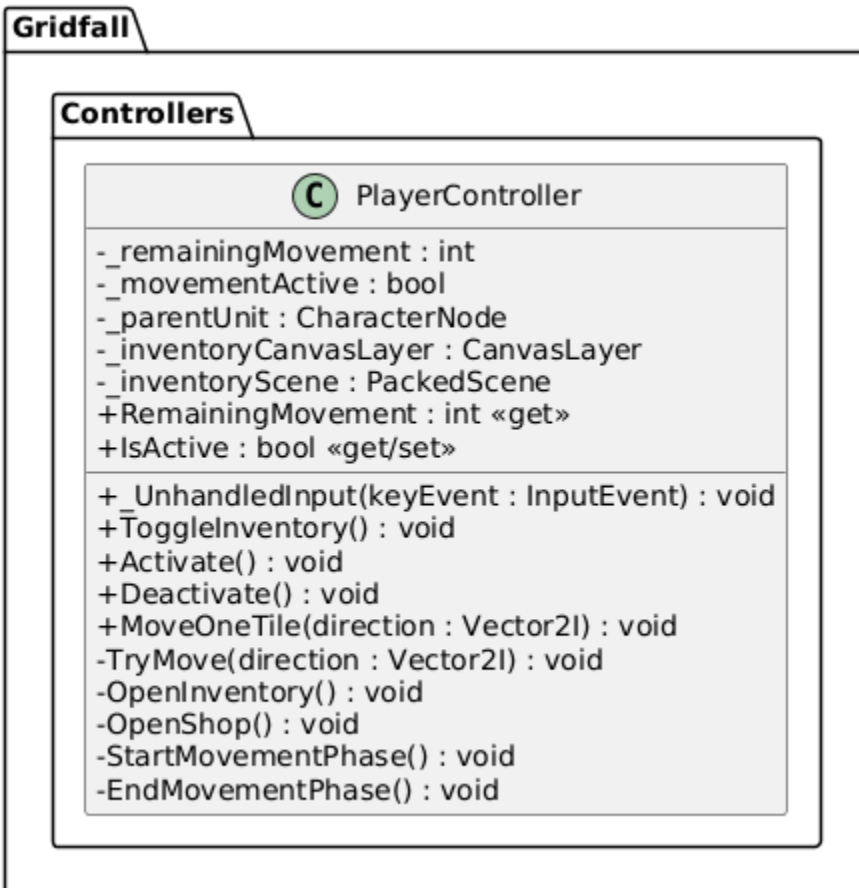
Gridfall implements an event-driven model of control flow centered around an interactive execution loop. Instead of following a linear procedural path where every user goes through the exact same steps in order, the application waits in a loop for user actions and system events to occur. The system listens for user inputs, such as directional movement keys, menu toggles, and combat interface buttons, as well as internal events like unit deaths or enemy spawns. Control flow moves through different gameplay phases such as player movement, turn-based combat, enemy turns, victory, and game over states in direct response to these events. This event-driven structure allows players to perform actions in different orders during their turn, giving them flexibility in how they interact with the game.

3. Detailed System Design

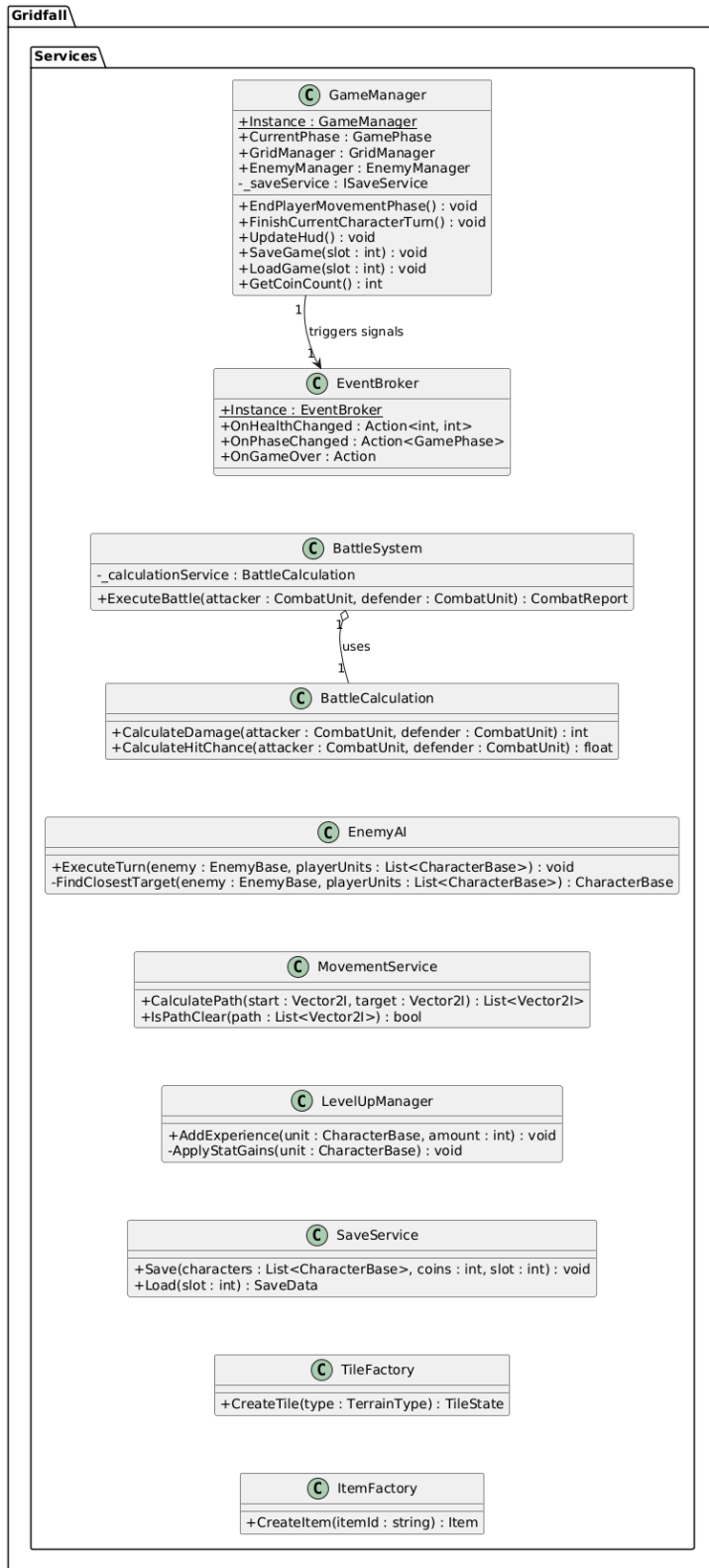
3.1 Static View

The below class UML diagrams show the static view of the classes within each package and their relations within each package.

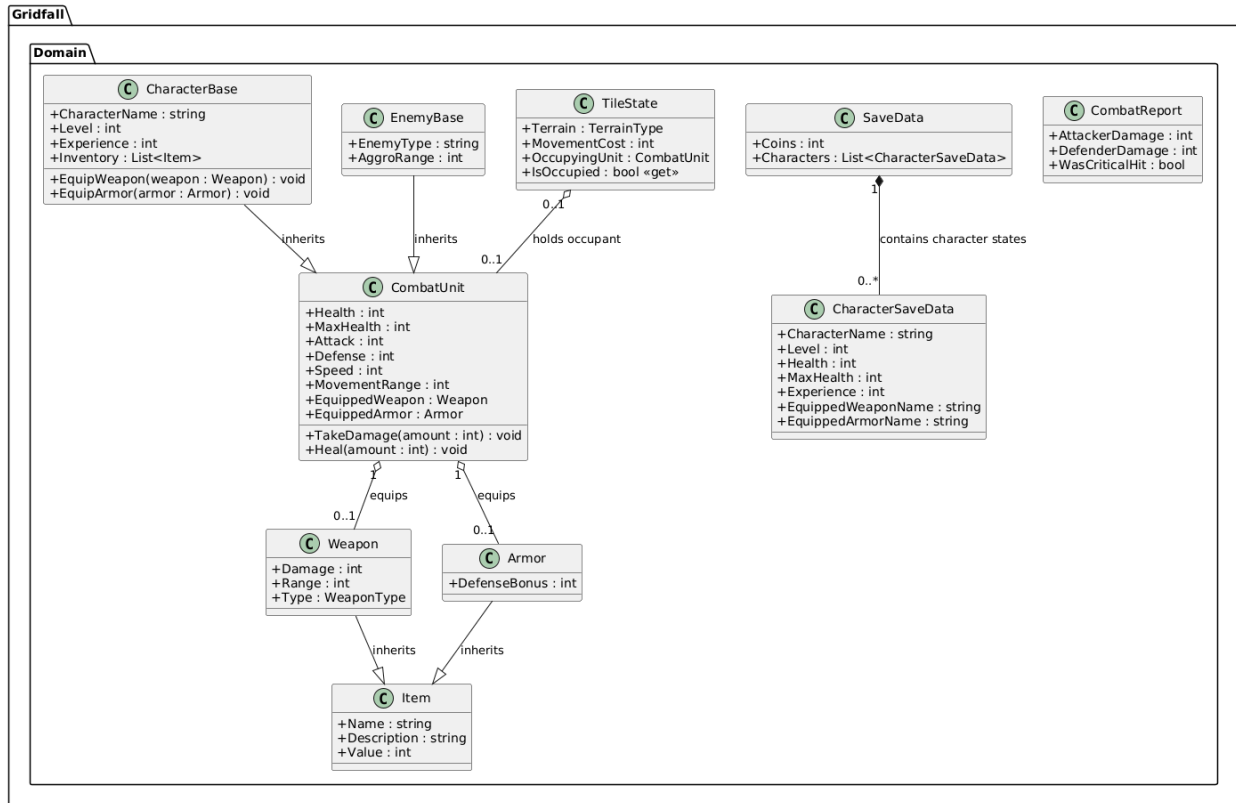
Controllers class diagram:



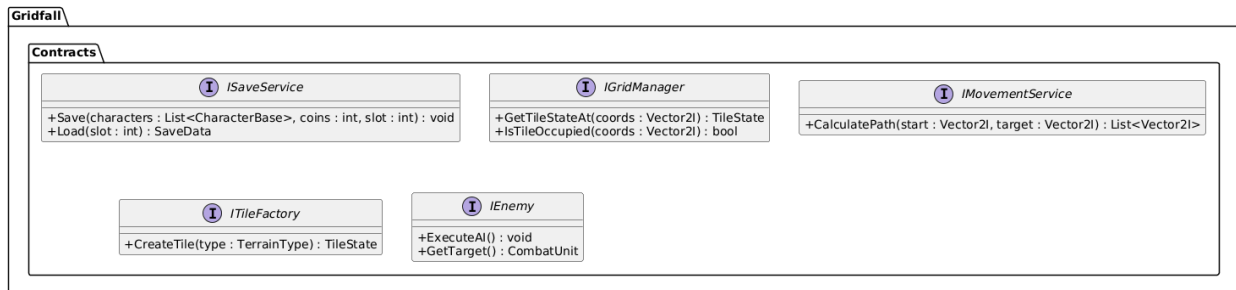
Services class diagram:



Domain class diagram:



Contracts class diagram:



The static view provides a clear decomposition of Gridfall's major packages.

Class responsibilities are divided among specialized components to maintain a clean separation of concerns. In Gridfall.Controllers, PlayerController handles keyboard input and maps user actions to character movement and menu controls. In Gridfall.Services, application logic is divided among dedicated managers and services: GameManager coordinates global phase

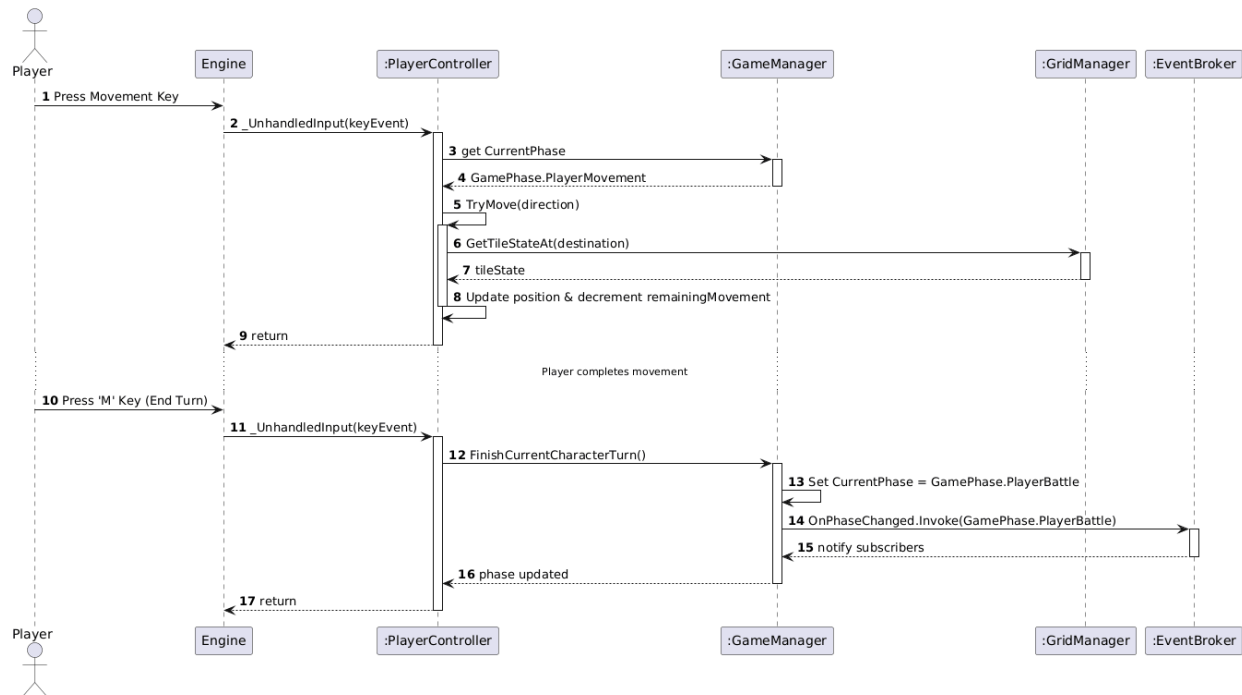
transitions and turn flow, BattleSystem and BattleCalculation evaluate combat damage, EnemyAI executes enemy decision-making, MovementService calculates grid pathfinding, LevelUpManager handles character experience progression, SaveService manages file reading and writing, TileFactory and ItemFactory create game objects, and EventBroker handles global signal events. In Gridfall.Domain, data structures represent core game concepts, including CombatUnit as the base model for player units represented by CharacterBase, and enemies, represented by EnemyBase, all containing unit statistics, TileState for terrain grid data tracking, Item as the base for Weapon and Armor, containing item statistics, and SaveData for save data structures. In Gridfall.Contracts, interfaces like ISaveService and IGridManager define system abstractions to keep modules decoupled.

Several software design patterns are applied to solve common architectural challenges. The Singleton pattern is used in GameManager and EventBroker to provide global access for phase coordination and event messaging. The Factory pattern is applied in TileFactory and ItemFactory to instantiate tiles and items dynamically based on type identifiers without coupling calling code to concrete constructors. The Observer pattern is implemented through EventBroker, enabling system components to broadcast events, such as unit deaths, so user interface displays can react automatically without requiring game entities to hold direct references to visual UI elements.

3.2 Dynamic view

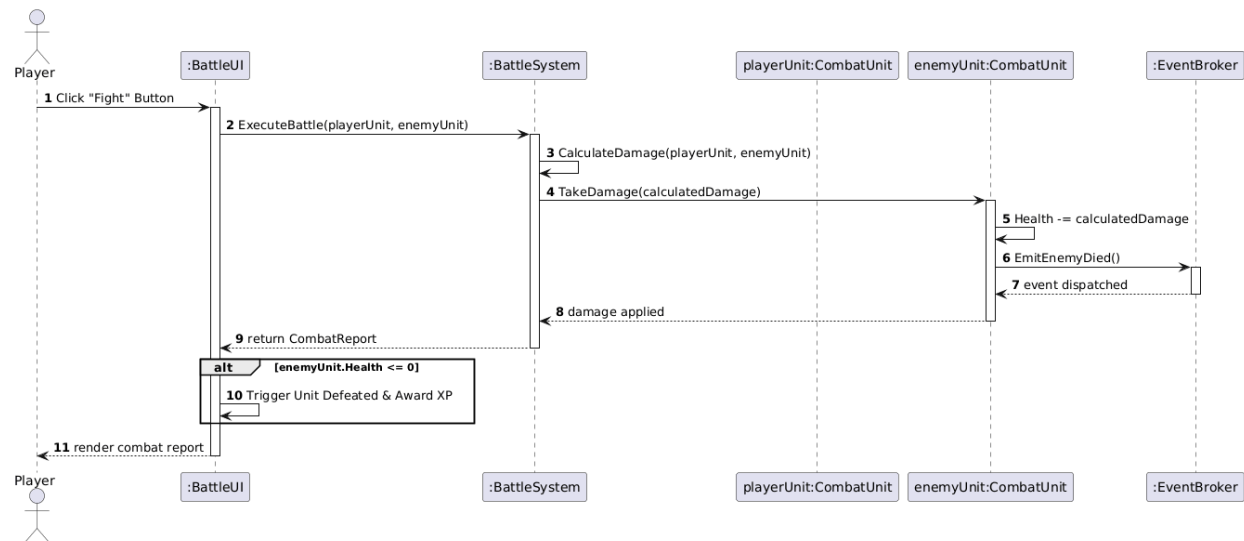
The following sequence diagrams show the dynamic view of the core functions within the game and packages. Some included participants are things not a part of the code base such as the Godot engine or disk. While not a class these are core parts of the game's sequences that must be interacted with to fulfill each sequence.

Movement and end turn sequence:



When the player presses a movement key, the engine sends the input event to `PlayerController._UnhandledInput()`. The controller queries `GameManager` to verify that the current phase is `PlayerMovement`. Upon verification, `PlayerController` executes `TryMove()`, calling `GridManager.GetTileStateAt()` to confirm that the target tile is reachable and unoccupied. Once validated, the unit's position is updated and movement points are decremented. When the player finishes moving and presses the end turn key, `PlayerController` invokes `GameManager.FinishCurrentCharacterTurn()`. `GameManager` updates its phase state machine to `PlayerBattle` and triggers `Events.OnPhaseChanged`, notifying subscribed UI components to display the battle interface.

Combat sequence:

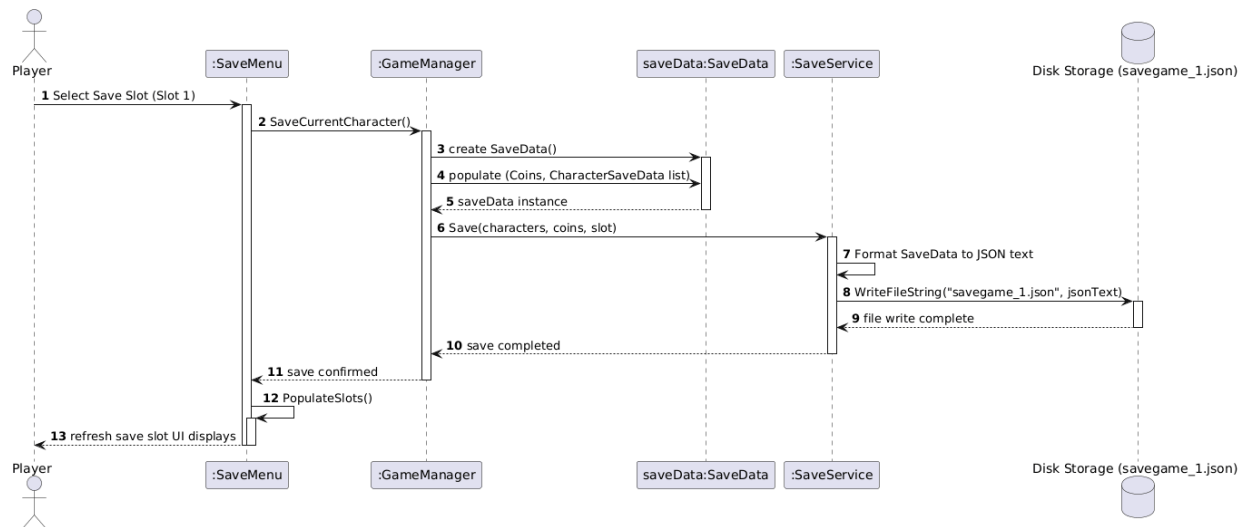


When the player confirms an attack by clicking Fight on the battle interface, the UI calls

BattleSystem.ExecuteBattle(), passing the attacking player unit and target enemy unit.

BattleSystem evaluates unit statistics using BattleCalculation to calculate total damage, then calls TakeDamage() on the defending unit. The defending unit subtracts the damage from its health attribute. If the unit's health drops to zero, it calls Events.EmitEnemyDied(), which notifies subscribed observers like GameManager to update enemy counts. Finally, BattleSystem returns a CombatReport to the battle UI, which renders the outcome and grants experience points to the player.

Save game sequence:



When the player opens the pause menu and clicks Save Game, the menu invokes `GameManager.SaveGame()`. `GameManager` gathers the player's coin balance and character unit statistics, constructing a `SaveData` transfer object populated with `CharacterSaveData` records. `GameManager` then passes the character list and coin count to `SaveService.Save()`. `SaveService` formats the object into structured JSON text and writes the file directly to host disk storage at `user://savegame_{slot}.json`. Once completed, `SaveService` confirms success back to `GameManager`, which updates the pause menu interface to display a confirmation message to the player.